

Tower of Hanoi

Recurrence Relation and Substitution Method

Contents

1	Tower of Hanoi	2
2	Recursive Idea	2
3	Base Condition	2
4	Solving the Recurrence Using Substitution Method	3
4.1	First Substitution	3
4.2	Second Substitution	3
4.3	Third Substitution	3
5	General Form After k Substitutions	4
6	Substitute $k = n - 1$	4
7	Solve the Geometric Series	4
8	Time Complexity	5
9	Complete Substitution in One View	5
10	Example	5
11	Quick Revision	6
12	Key Point	6

1 Tower of Hanoi

The Tower of Hanoi is a recursive problem involving n disks and three rods:

- Source
- Auxiliary
- Destination

The objective is to move all n disks from the Source rod to the Destination rod. The following rules must be followed:

1. Only one disk can be moved at a time.
2. A larger disk cannot be placed on a smaller disk.
3. Only the top disk of a rod can be moved.

2 Recursive Idea

To move n disks from Source to Destination:

1. Move $n - 1$ disks from Source to Auxiliary.
2. Move the largest disk from Source to Destination.
3. Move $n - 1$ disks from Auxiliary to Destination.

Therefore, the problem of size n requires:

2 problems of size $(n - 1)$

and one additional move.

Hence, the recurrence relation is:

$$T(n) = 2T(n - 1) + 1$$

3 Base Condition

For one disk, only one move is required.

Therefore:

$$T(1) = 1$$

Thus, the complete recurrence relation is:

$$\begin{aligned} T(n) &= 2T(n - 1) + 1 \\ T(1) &= 1 \end{aligned}$$

4 Solving the Recurrence Using Substitution Method

We have:

$$T(n) = 2T(n-1) + 1$$

We repeatedly substitute the recurrence relation.

4.1 First Substitution

We know:

$$T(n-1) = 2T(n-2) + 1$$

Substitute this into $T(n)$:

$$\begin{aligned} T(n) &= 2T(n-1) + 1 \\ &= 2[2T(n-2) + 1] + 1 \\ &= 2^2T(n-2) + 2 + 1 \end{aligned}$$

Therefore:

$$\boxed{T(n) = 2^2T(n-2) + 2 + 1}$$

4.2 Second Substitution

Now substitute:

$$T(n-2) = 2T(n-3) + 1$$

Therefore:

$$\begin{aligned} T(n) &= 2^2[2T(n-3) + 1] + 2 + 1 \\ &= 2^3T(n-3) + 2^2 + 2 + 1 \end{aligned}$$

Therefore:

$$\boxed{T(n) = 2^3T(n-3) + 2^2 + 2 + 1}$$

4.3 Third Substitution

Again,

$$T(n-3) = 2T(n-4) + 1$$

Therefore:

$$\begin{aligned} T(n) &= 2^3[2T(n-4) + 1] + 2^2 + 2 + 1 \\ &= 2^4T(n-4) + 2^3 + 2^2 + 2 + 1 \end{aligned}$$

Therefore:

$$\boxed{T(n) = 2^4T(n-4) + 2^3 + 2^2 + 2 + 1}$$

5 General Form After k Substitutions

From the above pattern, after k substitutions:

$$T(n) = 2^k T(n - k) + (2^{k-1} + 2^{k-2} + \cdots + 2^2 + 2 + 1)$$

We continue the substitution until we reach the base condition:

$$T(1) = 1$$

Therefore:

$$n - k = 1$$

Hence:

$$k = n - 1$$

6 Substitute $k = n - 1$

Substituting $k = n - 1$ into the general form:

$$T(n) = 2^{n-1} T(1) + (2^{n-2} + 2^{n-3} + \cdots + 2 + 1)$$

Since:

$$T(1) = 1$$

we get:

$$T(n) = 2^{n-1} + (2^{n-2} + 2^{n-3} + \cdots + 2 + 1)$$

7 Solve the Geometric Series

We know the geometric series:

$$1 + 2 + 2^2 + \cdots + 2^{n-2} = 2^{n-1} - 1$$

Therefore:

$$\begin{aligned} T(n) &= 2^{n-1} + (2^{n-1} - 1) \\ &= 2^n - 1 \end{aligned}$$

Hence, the exact number of moves required for the Tower of Hanoi is:

$$T(n) = 2^n - 1$$

8 Time Complexity

We obtained:

$$T(n) = 2^n - 1$$

For Big-O notation, constant terms are ignored.
Therefore:

$$T(n) = O(2^n)$$

Hence:

$$\boxed{\text{Time Complexity} = O(2^n)}$$

The Tower of Hanoi therefore has an **exponential time complexity**.

9 Complete Substitution in One View

The complete substitution can be summarized as:

$$\begin{aligned} T(n) &= 2T(n-1) + 1 \\ &= 2[2T(n-2) + 1] + 1 \\ &= 2^2T(n-2) + 2 + 1 \\ &= 2^2[2T(n-3) + 1] + 2 + 1 \\ &= 2^3T(n-3) + 2^2 + 2 + 1 \\ &= 2^4T(n-4) + 2^3 + 2^2 + 2 + 1 \\ &\vdots \\ &= 2^{n-1}T(1) + (2^{n-2} + 2^{n-3} + \dots + 2 + 1) \\ &= 2^{n-1} + (2^{n-1} - 1) \\ &= 2^n - 1 \end{aligned}$$

Therefore:

$$\boxed{T(n) = 2^n - 1}$$

and finally:

$$\boxed{O(2^n)}$$

10 Example

For $n = 3$ disks:

$$T(3) = 2^3 - 1$$

$$T(3) = 8 - 1 = 7$$

Therefore, 3 disks require:

$$\boxed{7 \text{ moves}}$$

Similarly, for $n = 4$:

$$T(4) = 2^4 - 1 = 15$$

For $n = 5$:

$$T(5) = 2^5 - 1 = 31$$

11 Quick Revision

Step	Result
Recurrence relation	$T(n) = 2T(n - 1) + 1$
Base condition	$T(1) = 1$
After k substitutions	$2^k T(n - k) + (2^{k-1} + \dots + 1)$
Reach base case	$k = n - 1$
Exact solution	$T(n) = 2^n - 1$
Big-O complexity	$\boxed{O(2^n)}$

12 Key Point

The most important idea to remember is:

$$\boxed{T(n) = 2T(n - 1) + 1}$$

Because every problem of size n creates two problems of size $n - 1$, the amount of work approximately doubles at every level.

Therefore:

$$\boxed{T(n) = 2^n - 1}$$

and the time complexity is:

$$\boxed{O(2^n)}$$